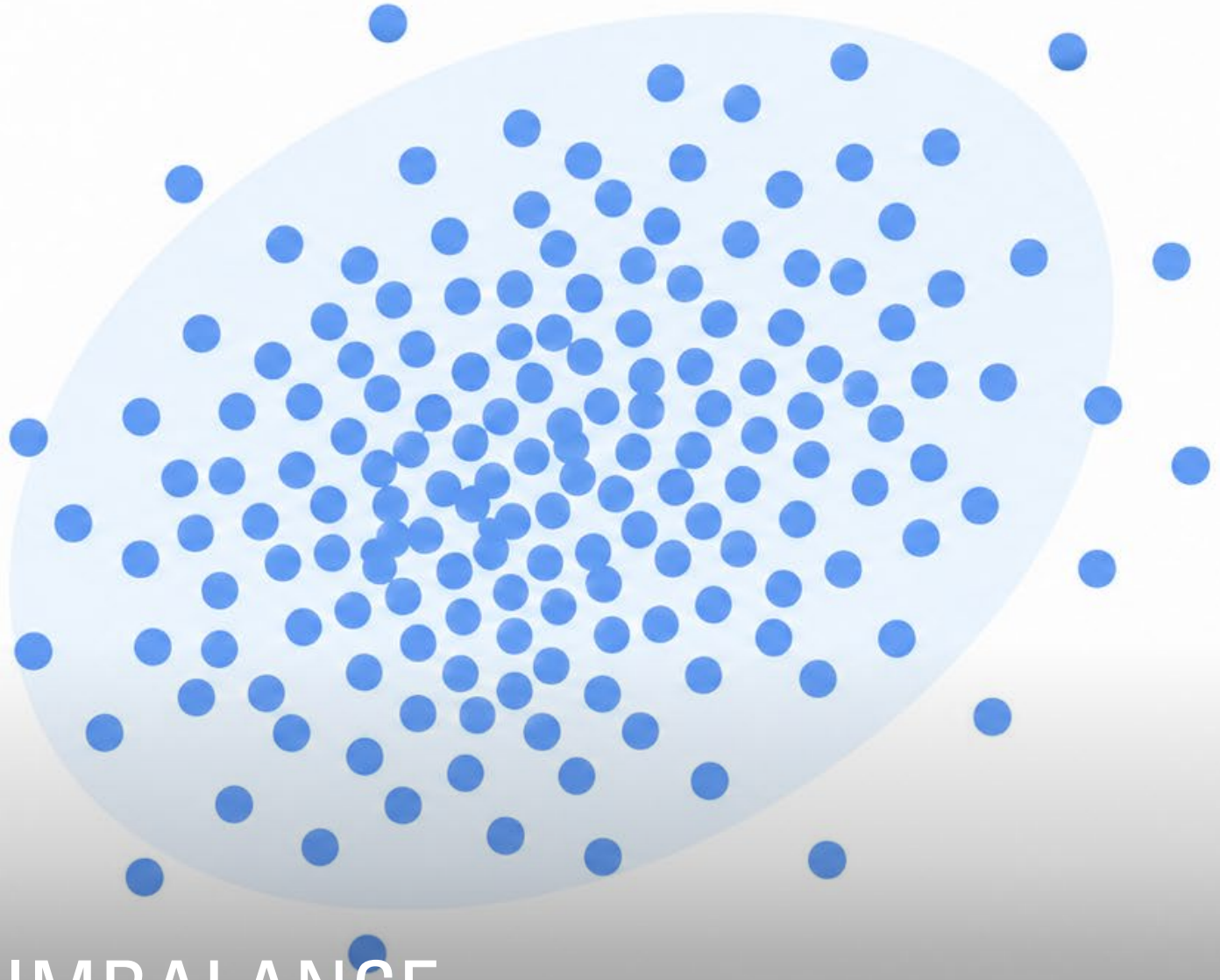
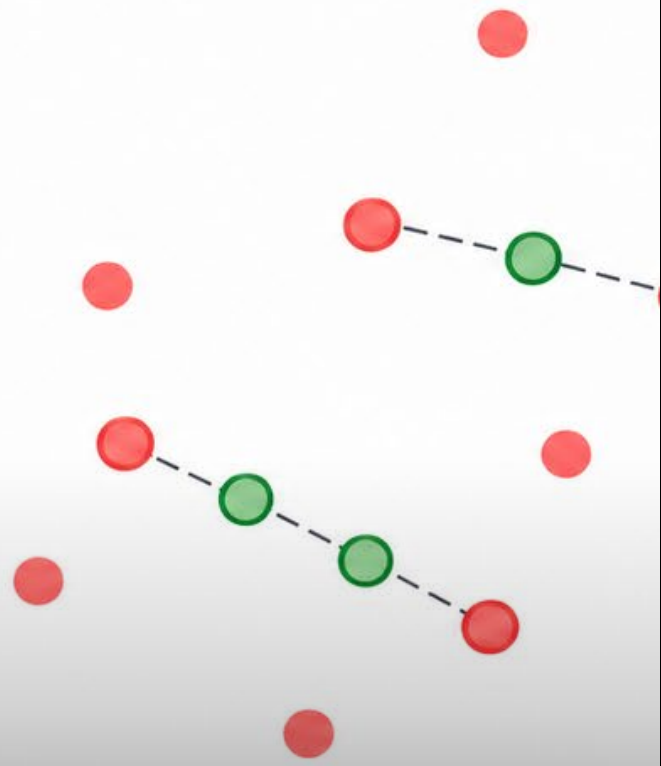




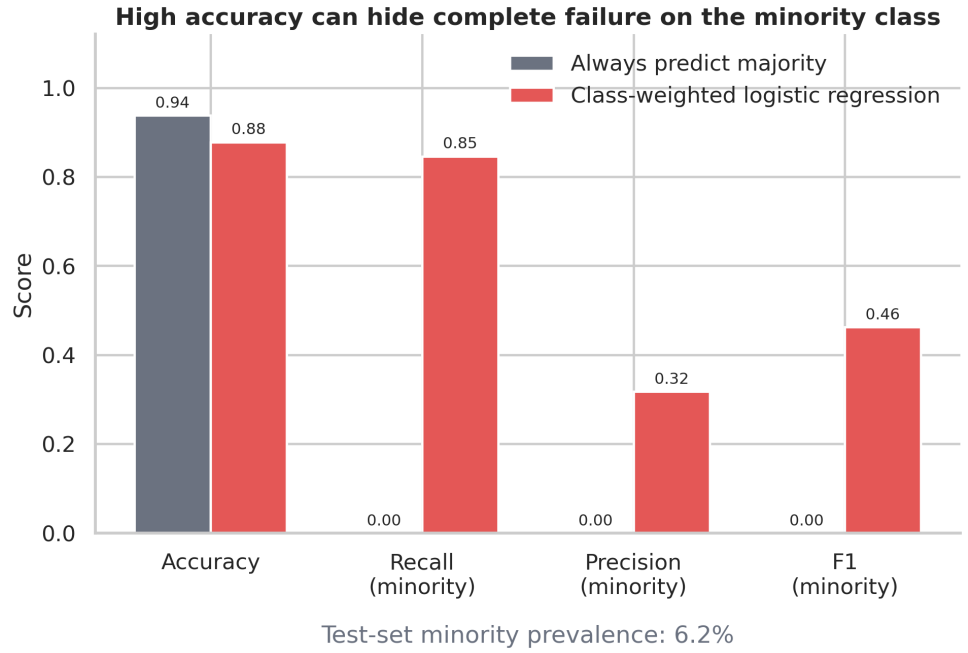
IMBALANCE

RARE OUTCOMES, IMPORTANT DECISIONS

- Class imbalance means one class is much rarer than another.
- The minority class often matters most, as in fraud or equipment failure.
- Imbalance can reflect the real world without indicating a data error.
- Learn minority patterns that remain useful in deployment.

Which costs more: a missed case or a false alarm?

HIGH ACCURACY CAN HIDE MISSED CASES



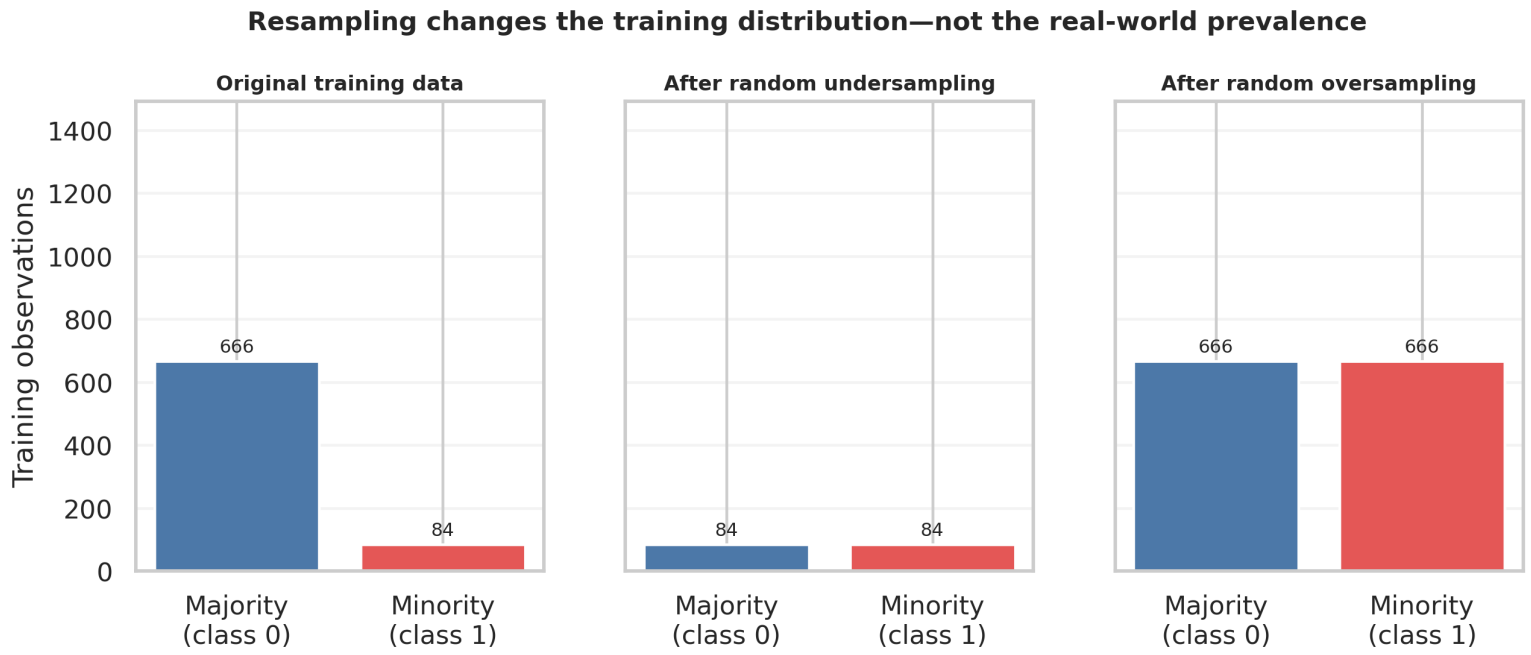
- With 95% negatives, always predicting negative gives 95% accuracy and 0% positive recall.
- Accuracy weights class performance by prevalence.
- A high AUC can still require threshold adjustment for a useful precision/recall tradeoff.

Illustrative figure: 6.2% minority prevalence.

Metrics: recall, precision, F1, balanced accuracy, PR-AUC, ROC-AUC, and expected cost.

ORIGINAL, TRAINING, AND DEPLOYMENT DATA

Resampling changes the training distribution.



Original

Observed prevalence and feature distribution.

Training

Rows or weights after the learning intervention.

Deployment

Keep test data untouched and representative of deployment.

ERROR COSTS GUIDE METRIC CHOICE

Which costs more: a missed case or a false alarm?

Missed cases cost more

Recall: share of positives detected.

Test a lower threshold.

Check false alarms and workload.

False alarms cost more

Precision: share of alerts that are true.

Test a higher threshold.

Check missed positives.

Both costs known? Compare expected error cost on validation data.

Review limit? Choose a cutoff within capacity, such as 200 alerts per day.

RANDOM UNDERSAMPLING

Remove majority-class rows

Benefit

Smaller training sets fit faster, reduce majority-class dominance, and may improve minority recall.

Tradeoff

Discarded majority cases may contain useful structure, especially near the class boundary.

Best fit: an extremely large, redundant majority class relative to computation needs.

RANDOM OVERSAMPLING

- Duplicate minority examples to balance training influence.
- Simple and model-agnostic. Keeps all majority observations.
- Duplicates can overfit flexible models or tiny minority samples.

Oversample only within each training fold.

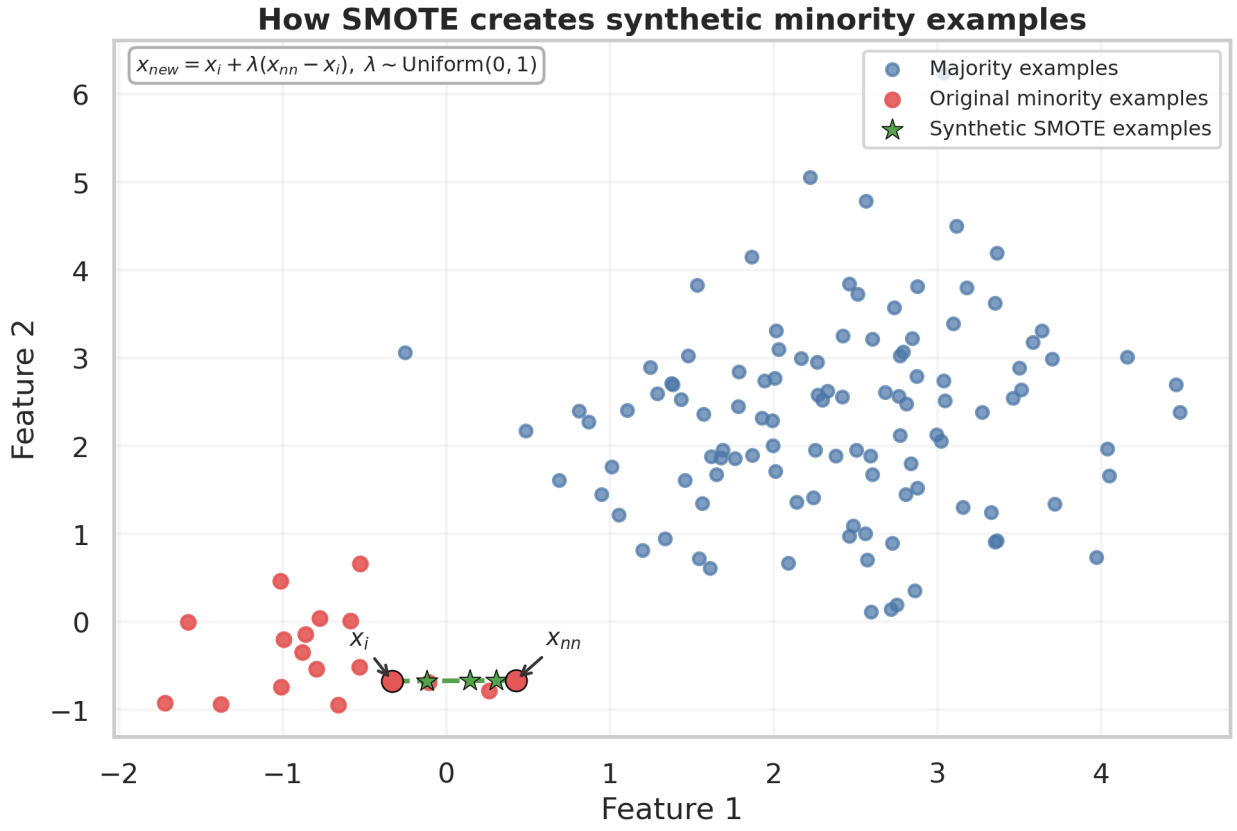
CLASS WEIGHTING AND ERROR COSTS

Increase the loss attached to minority-class errors without changing the rows.

$$L = -\sum_{i=1}^{n} [w_1 y_i \log(p_i) + w_0(1 - y_i) \log(1 - p_i)]$$

- Larger w_1 makes positive-class errors more influential during training.
- Use class_weight="balanced" or sample_weight where the estimator supports it.
- Keep all observations and avoid synthetic rows. Tune weights against the task, then validate calibration and threshold performance.

SMOTE: SYNTHETIC MINORITY EXAMPLES



- Interpolate minority neighbors to add variation and smooth sparse minority regions.
- The new point lies on the segment connecting the two examples.
- Interpolation must create plausible cases. Scale numeric features before distance-based synthesis.
- Use an appropriate method for categorical or mixed data.

SMOTE: INTERPOLATION MATHEMATICS

Let $x_i \in \mathbb{R}^d$ be a minority observation and x_j a minority neighbor.

$$\lambda \sim \text{Uniform}(0, 1)$$

$$z = x_i + \lambda(x_j - x_i) = (1 - \lambda)x_i + \lambda x_j$$

The synthetic point z is a convex combination of the two minority observations.

Illustrative calculation

$x_i = (1, 2), \quad x_j = (5, 6), \quad \lambda = 0.25$

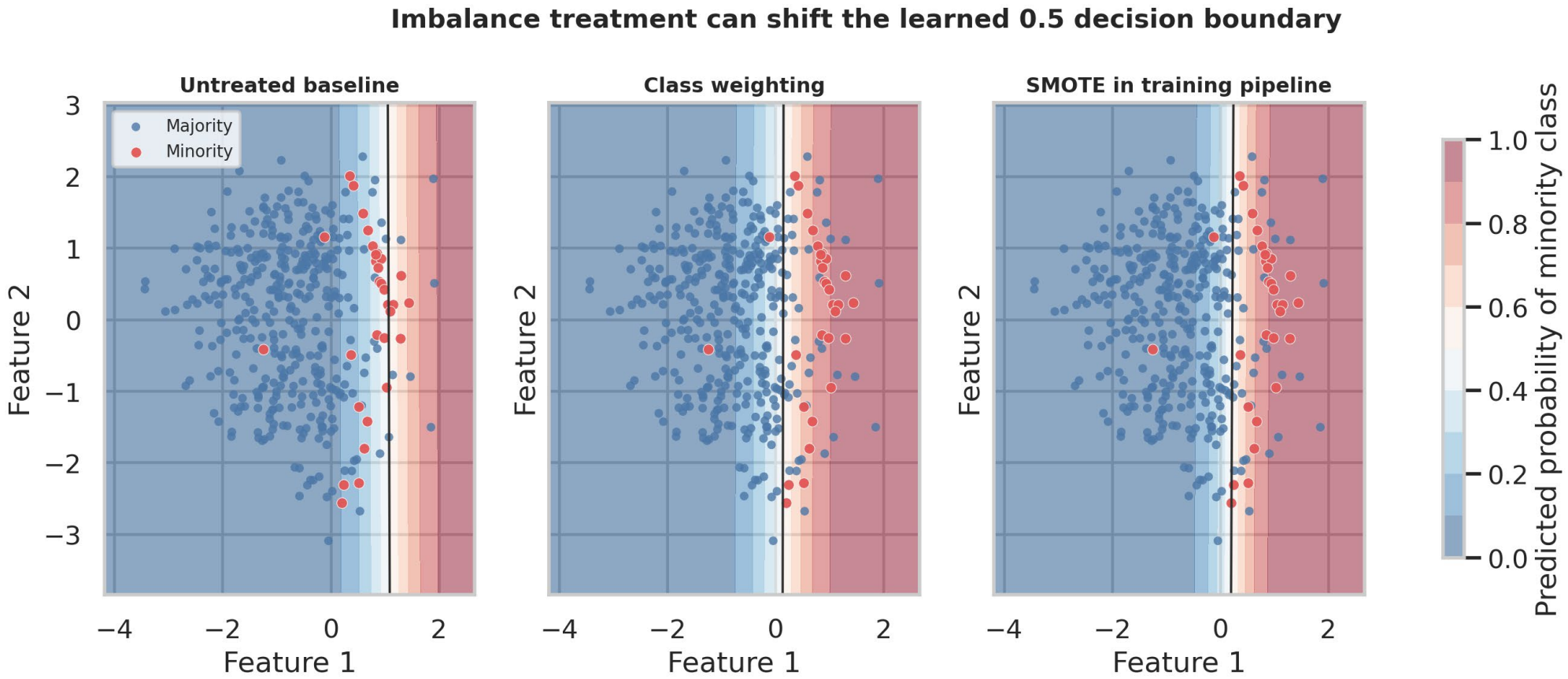
$z = (1, 2) + 0.25(4, 4) = (2, 3)$

Conditional mean

$$E[z \mid x_i, x_j] = \tfrac{1}{2}(x_i + x_j)$$

The midpoint of the selected pair.

DECISION BOUNDARIES AFTER TREATMENT



SMOTE VARIANTS

Borderline-SMOTE	Focuses on minority points near the decision boundary when boundary coverage matters.
ADASYN	Adds more samples in difficult minority regions, with a risk of amplifying noise.
KMeans-SMOTE	Clusters before synthesis, allocating samples to suitable regions when minority data form multiple clusters.
SMOTE-NC / SMOTEN	SMOTE-NC handles numeric and categorical features. SMOTEN handles categorical features.

Compare these alternatives against weighting and random oversampling.

CLEANING AND HYBRID APPROACHES

Tomek links

Find close opposite-class pairs. Removing selected examples can clean overlap near the boundary.

SMOTE + Tomek links: synthesis followed by overlap cleaning.

Edited Nearest Neighbors

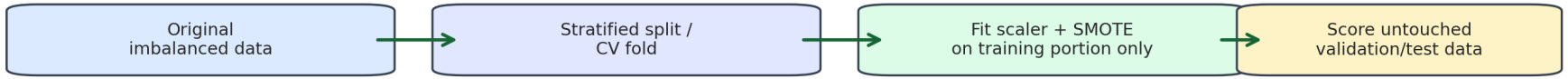
Remove examples whose labels disagree with their nearest neighbors to reduce local noise.

SMOTE + ENN: synthesis followed by stronger neighborhood cleaning.

Cleaning can remove legitimate difficult cases. Validate on untouched data.

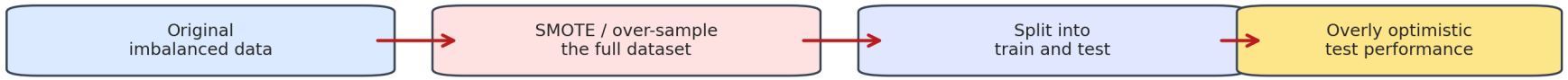
CORRECT AND INCORRECT RESAMPLING

Correct: split first; resample only within each training fold



No synthetic example or duplicate can enter the validation/test set.

Incorrect: resampling before the split leaks training information



Synthetic points or duplicates related to test observations may already influence training.

LEAKAGE-SAFE RESAMPLING

- Split training and test data before any learned preprocessing or resampling.
- Use stratified splits. Fit the scaler and resampler only on each fold's training partition.
- Score the untouched validation fold. Keep duplicates and synthetic descendants out of validation and test data.

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

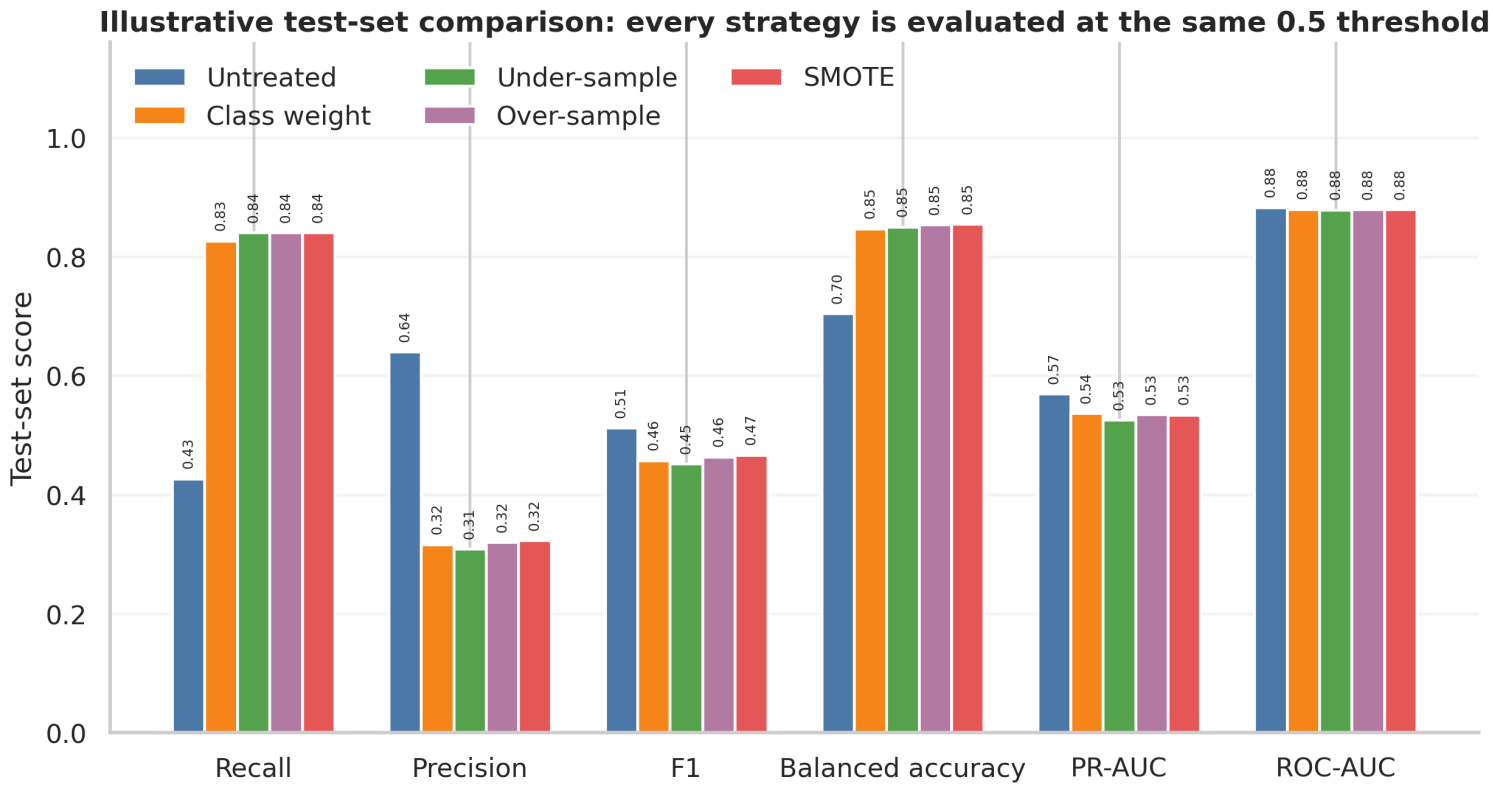

model = Pipeline([
    ("scale", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("classifier",
     LogisticRegression(max_iter=2000)),
])
```

Pass this pipeline to cross-validation.

METHOD COMPARISON FOR DEPLOYMENT

Method	Mechanism	Benefit	Risk / limitation	Best-use scenario
Random undersampling	Remove majority rows	Faster fitting, less dominance	Loses useful majority structure	Huge, redundant majority class
Random oversampling	Duplicate minority rows	Simple, keeps all majority rows	Duplicates may overfit	Quick baseline for small or medium data
Class weighting	Increase minority error loss	Keeps all original rows	Tune weights and calibration	First intervention for many classifiers
SMOTE	Interpolate minority neighbors	Adds nonduplicate variation	Implausible or overlapping cases	Numeric, sufficiently dense neighborhoods
Borderline-SMOTE	Synthesize near boundary	Emphasizes difficult boundary cases	Sensitive to noise and overlap	Boundary errors dominate
ADASYN	Synthesize in hard regions	Adapts to local difficulty	May amplify label noise	Uneven minority-region difficulty

PERFORMANCE AT A COMMON THRESHOLD



The test set retains the original minority prevalence; results depend on the simulated data and chosen threshold.

A PRACTICAL WORKFLOW

1 Define the decision	Specify the positive class, deployment prevalence, and error consequences.
2 Establish the baseline	Inspect recall, precision, PR-AUC, balanced accuracy, calibration, and thresholds.
3 Compare a small set	Compare thresholds, weighting, random over/undersampling, and one or two SMOTE-family methods.
4 Validate the full pipeline	Use stratified cross-validation and reserve an untouched final test set.

Choose the method and threshold that meet the deployment objective.